

AgroSearch — Motor de Busca Textual com Índice Invertido e TF-IDF

Gabriel Cavalcante Oliveira - Unipê - Ciência da computação

1. Visão geral

O **AgroSearch** é um protótipo de motor de busca textual em Streamlit (arquivo único **agrosearch.py**) para a AgroTech Solutions. O técnico digita uma consulta e recebe os trechos mais relevantes ranqueados por TF-IDF. Índice invertido, TF, IDF, TF-IDF e cosseno foram implementados do zero, com a biblioteca padrão do Python (**re**, **math**, **unicodedata**, **collections**); o pandas é usado apenas para exibir tabelas.

2. Arquitetura

Fase	O que foi implementado	Detalhes
1. Pré-processamento	Pipeline: tokenização → normalização → stopwords → stemming	Tokenização por regex sobre letras e dígitos; minúsculas e remoção de acentos via NFD; lista própria de stopwords em português, aplicada após a normalização; stemmer próprio por remoção de sufixos (plural, -ação, -mento, -dade, -ador, -ico e vogal final), com radical mínimo de 3 letras. Checkboxes na barra lateral ligam e desligam stopwords e stemming, e métricas comparam o tamanho do vocabulário nas quatro combinações. Ex.: irrigação, irrigar → irrig; lagartas → lagart.
2. Índice invertido	Dicionário termo → lista ordenada de IDs de documentos	Construído a partir dos tokens já limpos (cada documento aparece uma vez por termo). Ex.: soja → [1, 2, 4], irrig → [1, 5], lagart → [2, 4]. É exibido como tabela (com df e IDF) ou via st.json . Na busca, indica quais documentos contêm cada termo da consulta.
3. TF-IDF e ranqueamento	TF, IDF e score por documento	$TF(t,d) = f(t,d) / d $ (ou contagem bruta); $IDF(t) = \log_{10}(N / df(t))$ (ou versão suavizada); $Score(d) = \sum_{t \in q} TF(t,d) \cdot IDF(t)$. Documentos ordenados do maior para o menor score, com o vencedor destacado (troféu e caixa verde) e o trecho exibido. Termos fora do vocabulário são sinalizados, e há detalhamento de TF, IDF e TF-IDF por termo e documento.
Bônus: cosseno	Similaridade de cosseno entre vetores TF-IDF	A consulta e cada documento viram vetores TF-IDF esparsos e $\cos(q,d) = (q \cdot d) / (\ q\ \cdot \ d\)$. Isso normaliza pelo tamanho do documento e ajuda em consultas com várias palavras; o usuário pode ranquear por score ou por cosseno.
Interface	3 abas + barra lateral	Abas: Pré-processamento, Índice invertido e Busca/Ranking. Barra lateral com checkboxes de stopwords e stemming, escolha das variantes de TF/IDF, ativação do cosseno e base de documentos editável. As fórmulas ficam visíveis em st.latex .

3. Resultados de teste

Base de 5 documentos, TF relativo e IDF clássico.

Consulta	Termos processados	1º lugar	Score	Cosseno
irrigação da soja	irrig, soja	Doc 1	0,0775	0,257
controle de lagartas	control, lagart	Doc 2	0,1828	0,547
irrigar gotejamento	irrig, gotej	Doc 5	0,1567	0,458
água orgânico	agua, organ	Doc 5	0,1997	0,562

4. Análise

Na consulta “irrigação da soja”, o Doc 1 vence por conter os dois termos; soja (df = 3) tem IDF baixo e irrig (df = 2) pesa mais. Um termo presente em todos os documentos teria IDF clássico igual a 0, motivo pelo qual oferecemos o IDF suavizado.

5. Divisão de tarefas

Integrante	Responsabilidades
[Aluno 1]	Fase 1: tokenização, normalização, stopwords, stemmer; testes do pipeline.
[Aluno 2]	Fases 2 e 3: índice invertido, TF, IDF, TF-IDF, ranqueamento.
[Aluno 3]	Interface Streamlit, bônus de cosseno, relatório e revisão de código.

Execução: `pip install streamlit · streamlit run agrosearch.py`